Reviewer Pack — Second Eigenvalue Inverse Proportional to Resolution Proof L...

Entry 67d14839d492 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 08:44:26 UTC

Second Eigenvalue Inverse Proportional to Resolution Proof Length for Tseitin Formulas

Entry ID: `67d14839d492` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 08:44:26 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): Resolution Proof Length for Tseitin Formulas

Statement:

> For a random Tseitin formula derived from a d-regular expander graph with n nodes, the resolution proof length is Θ(1 / λ₂), where λ₂ is the second largest eigenvalue of the adjacency matrix. For all such instances with λ₂ ≤ 1/n, the proof length exceeds n^{1.5}.

Rationale (proposer's reasoning):

> Expander graphs' spectral properties govern their resistance to local reasoning. The second eigenvalue λ₂ quantifies the graph's expansion, and its inverse may correlate with the difficulty of deriving contradictions via resolution, as expanders require global interactions.

Taxonomy category: `PROOF_COMPLEXITY_TSEITIN` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `423b020fa375dfef`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def identity_matrix(n):
    I = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        I[i][i] = 1
    return I

def inverse(A):
    n = len(A)
    I = identity_matrix(n)
    for k in range(n):
        pivot = A[k][k]
        if pivot == 0:
            raise ValueError("Matrix is singular")
        for j in range(n):
            A[k][j] /= pivot
            I[k][j] /= pivot
        for i in range(n):
            if i != k:
                factor = A[i][k]
                for j in range(n):
                    A[i][j] -= factor * A[k][j]
                    I[i][j] -= factor * I[k][j]
    return I

def eigenvalues(A):
    n = len(A)
    lambda_values = []
```

```
    for _ in range(10):  # Simple power iteration method
        x = [random.random() for _ in range(n)]
        x_norm = sum(x[i] ** 2 for i in range(n)) ** 0.5
        x = [x_i / x_norm for x_i in x]
        for _ in range(10):
            y = matrix_multiplication(A, x)
            y_norm = sum(y[i] ** 2 for i in range(n)) ** 0.5
            y = [y_i / y_norm for y_i in y]
            lambda_new = sum(x[i] * y[i] for i in range(n))
            if abs(lambda_new - lambda_values[-1]) < 1e-6:
                break
        lambda_values.append(lambda_new)
    return sorted(lambda_values)

def tseitin_formula(G, root):
    n = len(G)
    literals = [f"v{i+1}" for i in range(n)]
    clauses = []
    for u in range(n):
        if G[u][root] == 1:
            clauses.append([literals[u]])
        else:
            clauses.append([-literals[u]])
        for v in range(u + 1, n):
            if G[u][v] == 1:
                clauses.append([literals[u], -literals[v]])
                clauses.append([-literals[u], literals[v]])
    return literals, clauses

def d_regular_expander(n, d):
    A = [[0 for _ in range(n)] for _ in range(n)]
    degree = [0] * n
    for u in range(n):
        while degree[u] < d:
            v = random.randint(0, n - 1)
            if u != v and degree[v] < d and A[u][v] == 0:
                A[u][v] = 1
                A[v][u] = 1
                degree[u] += 1
                degree[v] += 1
    return A

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    d = 2 * n // (n - 1)
    G = d_regular_expander(n, d)
    lambda_2 = eigenvalues(G)[1]
    literals, clauses = tseitin_formula(G, 0)

    def sat_solver(clauses):
        assignment = [False] * len(literals)
        stack = []
        for clause in clauses:
            found_unassigned = False
            for literal in clause:
```

```
                var_index = abs(literal) - 1
                if not assignment[var_index]:
                    assignment[var_index] = literal > 0
                    stack.append((var_index, literal))
                    found_unassigned = True
                    break
            if not found_unassigned:
                return False
        while stack:
            var_index, literal = stack.pop()
            for clause in clauses:
                if literal in clause and -literal in clause:
                    return False
        return True

    proof_length = 0
    for _ in range(10):  # Sample multiple instances
        if not sat_solver(clauses):
            proof_length += 1

    conjecture_holds = lambda_2 * proof_length > 1.5 * n ** 1.5
    counterexample = "" if conjecture_holds else f"n={n}, d={d}, λ₂={lambda_2}, proof_length={proo

    return {
        "metric_name": "Proof Length",
        "metric_value": proof_length,
        "instances_tested": 10,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results)) *
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3aa44c0b.py", line 147, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3aa44c0b.py", line 101, in run_trial
    lambda_2 = eigenvalues(G)[1]
               ^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3aa44c0b.py", line 59, in eigenvalues
    y = matrix_multiplication(A, x)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3aa44c0b.py", line 24, in matrix_multip
    C[i][j] += A[i][k] * B[k][j]
                         ~~~~^^^
TypeError: 'float' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to type error in matrix multiplication, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix matrix multiplication to handle vectors vs matrices correctly and retest with validated input types

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107231
2	preregistration	ollama_remote	qwen3:8b	0	0	24192
3	novelty	ollama_remote	qwen3:8b	0	0	20736
4	novelty	ollama_remote	qwen3:8b	0	0	16375
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22339
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15742
7	judge	ollama_remote	qwen3:8b	0	0	21342

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 227956 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/67d14839d492.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/67d14839d492.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/67d14839d492.tar.gz` (if generated)